

Fixed-Point Arithmetic & Quantisation

L7 · Week 5 · Hardware / Software Codesign 2025/26

José T. de Sousa · IST

Range

integer bits I

Precision

fractional bits $W-I$

Quantisation

SNR / RMSE

ap_fixed

methodology

How many bits are enough — and how do you know?

Where We Are

L5 · DMA

bulk transfer, HP ports

CPU-free data movement to/from PL

L6 · HLS Opt

II, pragmas, `ap_fixed` intro

PIPELINE $\rightarrow$ II=1; float costs 4 $\times$; `ap_fixed` r

L7 — Today · Fixed-Point Arithmetic & Quantisation: *how to choose W and I*

*L6 gave us the `ap_fixed` syntax and showed it recovers II=1. Today we answer the harder question: **how do you choose W and I for your algorithm?***

Today's Question

“How many bits are enough?”

Too few $\Rightarrow$ overflow or unacceptable rounding error. Too many $\Rightarrow$ wasted LUTs and DSPs.

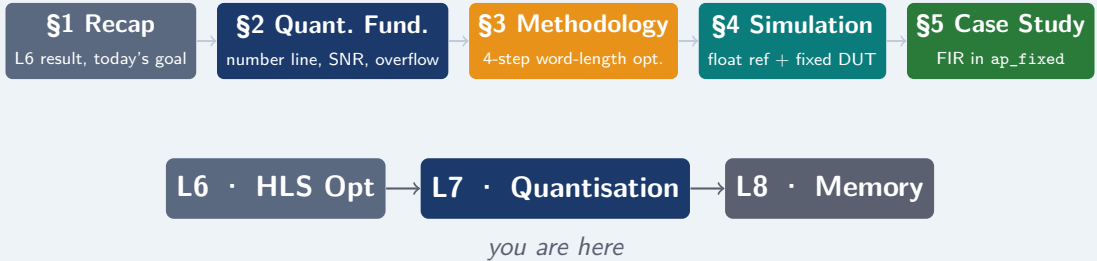
What we already know (L6)

- ▶ `ap_fixed<W,I>` syntax and types
- ▶ Fixed-point gives $II = 1$ vs. float $II = 4$
- ▶ Area & accuracy trade-off exists

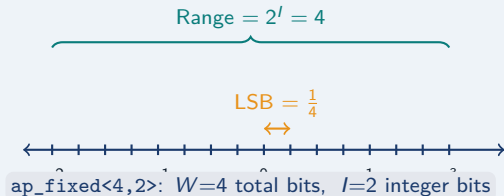
What L7 adds

- ▶ Quantisation error: how to measure it
- ▶ Word-length optimisation methodology
- ▶ Fixed-point C++ simulation & testbench
- ▶ Case study: FIR filter in `ap_fixed`

Lecture Roadmap



The Fixed-Point Number Line



Two parameters control everything:

- ▶ **I (integer bits)**: range $[-2^{I-1}, 2^{I-1}-\text{LSB}]$
- ▶ **$W-I$ (frac. bits)**: $\text{LSB} = 2^{I-W}$
- ▶ **Max error** $\leq \frac{\text{LSB}}{2}$ (with rounding)
- ▶ Each extra frac. bit **halves** error (+1 bit of hardware)

Goal: smallest W meeting your error target.

Truncation vs. Rounding

AP_TRN — Truncation (default)

Drop the extra bits — equivalent to floor.

- ▶ Error range: $[-\text{LSB}, 0)$
- ▶ **Biased** — average error $= -\frac{\text{LSB}}{2}$
- ▶ Free in hardware: no extra logic

AP_RND — Round to Nearest

Add $\frac{\text{LSB}}{2}$ before truncating.

- ▶ Error range: $[-\frac{\text{LSB}}{2}, +\frac{\text{LSB}}{2})$
- ▶ **Unbiased** — zero mean error
- ▶ Costs one adder in hardware

```
ap_fixed<8,4, AP_TRN> a = 3.17f; // Stored as 3.125
ap_fixed<8,4, AP_RND> b = 3.17f; // Stored as 3.1875
```

Rule of thumb: use AP_RND unless area is critical — the extra adder is negligible compared to the accuracy gain.

Overflow and Saturation

AP_WRAP — Wrap-around

Modular arithmetic: value wraps from MAX to MIN.

- ▶ Free: discard the carry-out bit
- ▶ Catastrophic if overflow is unexpected
- ▶ Use only when overflow is *provably impossible*

AP_SAT — Saturate

Clamp to [MIN, MAX] on overflow.

- ▶ Costs a comparator and mux
- ▶ Graceful degradation — no sign flip
- ▶ Required for audio, image, control loops

```
ap_fixed<8,4, AP_RND, AP_WRAP> w;  w = 7.9f;  // wraps to -8
ap_fixed<8,4, AP_RND, AP_SAT>  s;  s = 7.9f;  // clamps to
7.875
```

Default is AP_WRAP. Always use AP_SAT for outputs that feed audio codecs, pixel pipelines, or PID controllers.

Full ap_fixed Template

`ap_fixed<W, I, Q, 0>`

total bits

integer bits

quantise mode

overflow mode

Param	Common values	Meaning
W	8, 12, 16, 24	Total word width in bits
I	1, 2, 4, ...	Integer bits (including sign); sets range
Q	AP_TRN, AP_RND, AP_RND_CONV	What to do when dropping LSBs (conv. = banker's rounding)
0	AP_WRAP, AP_SAT	What to do when value exceeds range

// Typical project typedefs

```
typedef ap_fixed<16, 2, AP_RND, AP_SAT> pixel_t;    // image [-2,2)
typedef ap_fixed<16, 1, AP_RND, AP_SAT> audio_t;    // audio [-1,1)
typedef ap_fixed<24, 5, AP_RND, AP_SAT> accum_t;    // accumulator
```

Signed (ap_fixed) is the default. Use ap_ufixed for quantities that are always non-negative.

Quantisation Noise and SNR

Signal-to-Noise Ratio (SNR)

$$\text{SNR} = 10 \log_{10} \left(\frac{\sigma_{\text{signal}}^2}{\sigma_{\text{noise}}^2} \right) \text{ [dB]}$$

For **uniform quantisation with rounding**:

$$\sigma_{\text{noise}}^2 = \frac{\text{LSB}^2}{12} \Rightarrow \sigma_{\text{noise}}^2 = \frac{2^{2(I-W)}}{12}$$

For a full-scale sinusoid ($\sigma_s^2 = A^2/2$):

$$\text{SNR} \approx 6.02 (W - I) + 1.76 \text{ dB}$$

Each extra fractional bit adds ≈ 6 dB.

Typical targets:

Application	Target	Min bits
Studio audio	~ 144 dB	24
CD audio	~ 96 dB	16
Image proc.	40 dB	7
Control loop	30 dB	5

In practice: always *measure* SNR in a C test-bench with real data. The formula assumes uniform signal distribution — your data may differ.

Common ap_fixed Pitfalls

Pitfall 1 — Accumulator overflow

After N multiply-accumulates, the result can be $N\times$ larger than one term. Add $\lceil \log_2 N \rceil$ extra integer bits:

```
// 16-tap FIR, data in [-1,1)
// product in [-1,1): I=1
// sum of 16: max=16, need
// I=5
typedef ap_fixed<16,1>
    data_t;
typedef ap_fixed<20,5>
    acc_t; // +4 bits
```

Pitfall 2 — Silent narrowing

Assigning a wide type to a narrow one silently drops bits. HLS will not error — check the synthesis log:

```
acc_t acc = compute(); //
    wide
data_t out = acc; // SILENT
    truncation
// Fix: explicit cast or
    resize
data_t out = (data_t)acc;
```

Rule: after writing your HLS function, search the Vitis HLS log for WARNING: [XFORM] width-change messages before trusting the output.

Section 3 — Word-Length Optimisation Methodology

We know what ap_fixed parameters mean. Now: a systematic way to choose them.



- ▶ **Step 1:** profile your float reference to find the dynamic range
- ▶ **Step 2:** define the accuracy requirement for your application
- ▶ **Step 3:** sweep W in C simulation, measure SNR or RMSE
- ▶ **Step 4:** synthesise the chosen W ; verify $II = 1$ and area budget

Step 1 — Sizing the Integer Part (/ bits)

Find the dynamic range first:

```
// Run float reference; record extremes
float max_abs = 0;
for (int n = 0; n < N; n++) {
    float v = compute_float(x,
        n);
    if (fabsf(v) > max_abs)
        max_abs = fabsf(v);
}
// Size I to cover max_abs with margin
int I =
    (int)ceilf(log2f(max_abs))
    + 2;
```

Formula:

$$I = \lceil \log_2 V_{\max} \rceil + 1 \text{ (sign)} + 1 \text{ (guard)}$$

Examples:

Signal	$V_{\max}$	I
Audio $[-1, 1]$	1	2
GEMM row ($n=64$)	64	8
FIR output ($N=16$)	16	6

Always add **1–2 guard bits** above the measured maximum. Corner-case inputs may exceed your training set.

Word-Length Methodology: Vary FIN in C-sim

Signal representations derived from IIN:

Signal	Format
Input / coeff	ap_fixed<WIN,IIN>
Product	ap_fixed<2*WIN,2*IIN>
Accumulator	ap_fixed<2*WIN+G,2*IIN+G>

Change #define FIN and rerun

csim_design.

Measure max error vs. double baseline.

Choose smallest FIN meeting error target.

```
#define IIN 1 // range [-1,1)
#define FIN 12 // vary this
#define WIN (FIN+IIN)
#define G 4 // log2(16 taps)
typedef ap_fixed<WIN, IIN>
    data_t;
typedef ap_fixed<2*WIN, 2*IIN>
    mul_t;
typedef
    ap_fixed<2*WIN+G,2*IIN+G>
    acc_t;
// testbench: compare with float
// ref
double err = fabs(hw_y - ref_y);
```

ap_fixed Arithmetic — Format Propagation Rules

When two *ap_fixed* values are combined, the result format follows these rules. HLS applies them automatically.

Operation	Integer bits	Frac. bits	Example
Add/Sub $R = A \pm B$	$I_R = \max(I_A, I_B) + 1$	$F_R = \max(F_A, F_B)$	$\langle 7, 3 \rangle + \langle 5, 2 \rangle \rightarrow \langle 8, 4 \rangle$
Multiply $M = A \times B$	$I_M = I_A + I_B$	$F_M = F_A + F_B$	$\langle 7, 3 \rangle * \langle 5, 2 \rangle \rightarrow \langle 12, 5 \rangle$
Divide $D = A / B$	$I_D = I_A + F_B$	$F_D = F_A$	$\langle 7, 3 \rangle / \langle 5, 2 \rangle \rightarrow \langle 9, 3 \rangle$

Note: division requires extending dividend *A* before the operation.

Key insight: multiplication grows the word fast — `ap_fixed<12,1> × ap_fixed<12,1>` needs a `<24,2>` result. Always accumulate into a wider type.

ap_fixed Bit and Range Operators

Bit operator [i]

Selects one bit. LSB has index 0. Returns a reference — can read or write.

```
ap_fixed<8,5> v = 1.375; //  
    0b00001.011  
v[3];           // 1  (bit 3 is set)  
v[4];           // 0  (bit 4 is clear)  
v[2] = 1;       // v becomes 1.875  
v[3] = 0;       // v becomes 0.875
```

Range operator (Hi,Lo)

Selects a group of bits. If $Lo > Hi$, bits are returned in reverse order.

```
ap_ufixed<4,2> u = 1.75; // 0b01.11  
ap_uint<4> r, rev;  
r = u.range(3,0); // 7  (0b0111)  
rev = u.range(0,3); // 14 (0b1110)  
// Assign a slice from another  
// variable:  
ap_fixed<4,2> s;  
ap_fixed<8,0> x = 0xAE;  
s(3,0) = x(5,2); // s = -1.25
```

ap_fixed Console Output — to_string() and c_str()

For C-simulation, HLS supports printing ap_fixed values in binary, decimal, or hexadecimal — including values wider than 64 bits.

```
ap_fixed<40,8> val = 112.1;
printf("binary: %s\n",
      val.to_string().c_str());
printf("decimal: %s\n",
      val.to_string(10).c_str());
printf("hexa: %s\n",
      val.to_string(16).c_str());
```

binary: 0b01110000.000110011...

decimal: 112.0999999998603...

hexa: 0x70.19999999p0

```
// Signed/unsigned control:
ap_fixed<8,4> v = -5.25;
printf("%s\n",
      v.to_string(2).c_str());
// 0b11010.1100
printf("%s\n",
      v.to_string(10).c_str());
// -5.25
printf("%s\n",
      v.to_string(16, true).c_str());
// -0x5.4p0 (signed)
printf("%s\n",
      v.to_string(16, false).c_str());
// 0x1a.cp0 (unsigned)
```

Use to_string() only in testbenches — not in synthesisable HLS code. These calls are simulation-only.

Step 4 — Synthesis Check

Once the C simulation gives a passing W , run HLS synthesis and verify three things:

II still = 1?

Wider types need more DSP slices. If the Zynq runs out of DSPs, HLS serialises multiplications and II rises. Fix: reduce W by 2 and re-check.

Area within budget?

Check the utilisation report: LUT, FF, DSP, BRAM. A $2\times$ increase in W roughly doubles DSP usage for multiply-heavy kernels.

Timing closed?

Wider adders have longer critical paths. Confirm the post-synthesis report shows ≥ 100 MHz. If not, reduce W or add a pipeline stage.

The trade-off curve: as W grows, SNR improves (good) but area and possibly II worsen (bad). Your job is to find the *knee* of that curve.

Practical Heuristics

- ▶ **Accumulator width** after N additions:

$$W_{\text{acc}} = W_{\text{data}} + \lceil \log_2 N \rceil \quad I_{\text{acc}} = I_{\text{data}} + \lceil \log_2 N \rceil$$

Example: 16-tap FIR, data `ap_fixed<12,2>` $\Rightarrow$ accumulator `ap_fixed<16,6>`

- ▶ **Multiplier output width** = $W_A + W_B$ bits. Always store in a type at least that wide before truncating.
- ▶ **Coefficients** can usually be 2–4 bits narrower than the data type without significant SNR loss (they are constant and bounded).
- ▶ **Start wide, narrow down.** Begin with $W = 24$ to confirm algorithm correctness, then reduce until SNR just meets the target.
- ▶ **Read the HLS warnings.** WARNING: [XFORM] `ap_fixed` width-change messages flag every silent truncation in your design.

Section 4 — Fixed-Point C++ Simulation

Step 3 of the methodology requires simulating your fixed-point design in C++. Here is the pattern that makes it reliable.

Float reference

The ground truth. Implemented in `float` or `double`. Known correct. You already have this from the SW phase.

ap_fixed DUT

The design under test. Same algorithm, types replaced with `ap_fixed<W,I,...>`. Run on the same inputs. Compare outputs to the reference.

- ▶ Run both on the **same inputs**: synthetic vectors, real data, or edge cases
- ▶ Compute **SNR or RMSE** from the difference
- ▶ Iterate W in a loop — C simulation is fast, no synthesis needed

Testbench Code Pattern

```
#include "ap_fixed.h"
#include <math.h>
#define N 256
typedef ap_fixed<16,2, AP_RND, AP_SAT> fx_t;
float compute_float(float x); // float reference
fx_t compute_fixed(fx_t x); // ap_fixed DUT
int main() {
    float x_f[N];  fx_t x_fx[N];
    double err_sq = 0, sig_sq = 0;
    for (int n = 0; n < N; n++) {
        float ref = compute_float(x_f[n]);
        float dut = (float)compute_fixed(x_fx[n]);
        err_sq += (ref-dut)*(ref-dut);
        sig_sq += ref*ref;
    }
    float snr = 10.0f*log10f((float)(sig_sq/err_sq));
    printf("SNR = %.1f dB\n", snr);
    return (snr >= 60.0f) ? 0 : 1;
}
```

Scale-and-Round: Sweeping W Without Recompile

Quantise in pure C by multiplying by $S = 2^N$, rounding, then dividing back:

```
float quantize(float x, float S) {  
    return roundf(x * S) / S;  
}
```

Sweep all scale factors in one run — no recompilation needed:

```
for (int N = 4; N <= 20; N++) {  
    float S = powf(2.0f, (float)N);  
    double err_sq = 0, sig_sq = 0;  
    for (int i = 0; i < LEN; i++) {  
        float q = quantize(ref[i], S);  
        err_sq += (ref[i]-q)*(ref[i]-q);  
        sig_sq += ref[i]*ref[i];  
    }  
    float snr = 10.0f*log10f((float)(sig_sq/err_sq));  
    printf("N=%2d SNR=%5.1f dB\n", N, snr);  
}
```

1. No `ap_fixed` type needed — works with any float reference
2. One executable sweeps all N values in seconds

C-Simulation vs. Co-Simulation

Property	C-Simulation	Co-Simulation (RTL)
What runs	Pure C++, no HW model	Generated RTL via Model-Sim/Xsim
Speed	Sub-second	Minutes (RTL compile + simulate)
What it checks	Algorithm, data types, SNR	AXI interface, II, pipeline stalls
When to use	Word-length sweeping	Final sign-off before export
Verdict	Use for W exploration	Use once W is chosen
Workflow: run C-sim in a loop to find W (fast, seconds per point) → then run co-sim once on the final design to confirm the RTL behaves identically to the C model.		

Debugging — Bisecting W

SNR not improving with wider W ?

Follow this procedure:

1. Check testbench for overflow
 - **Overflow detected:** increase I , keep $W-I$ fixed
 - **No overflow:** increase $W-I$ (more fractional bits)
2. Re-run csim and measure SNR again
3. Repeat until SNR meets target

How to detect overflow:

```
// Before the simulation loop:
ap_fixed<W,I, AP_RND, AP_SAT>
    v;
v = huge_value;
if ((float)v >= (1<<(I-1)) -
    LSB) {
    printf("OVERFLOW at I=%d\n", I);
}
```

A sharp SNR drop when increasing W usually means overflow, not precision. Increasing fractional bits cannot fix an overflow — you must increase I first.

Section 5 — Case Study: FIR Filter in `ap_fixed`

We apply the 4-step methodology end-to-end to a 16-tap FIR lowpass filter. Every fixed-point design challenge appears in this one example.

Why FIR is the ideal case study:

- ▶ Fixed coefficients — coefficient quantisation is a separate tunable knob
- ▶ Accumulator grows with N — forces the guard-bit calculation
- ▶ Output quality is easy to measure — frequency response or SNR
- ▶ Maps directly to HLS with PIPELINE
 $II = 1$

Filter spec:

Parameter	Value
Taps N	16
Input range	$[-1, 1)$
Coefficients	$[-1, 1)$
SNR target	60 dB
II target	1

Word-Length Analysis — 16-tap FIR

Signal	Range	Integer bits /
Input $x[n]$	$[-1, 1)$	$l_x = 1$ (sign only)
Coefficient $h[k]$	$[-1, 1)$	$l_h = 1$
Product $h[k] \cdot x[n-k]$	$(-1, 1)$	$l_p = l_x + l_h = 2$
Accumulator y	$\leq N \cdot 1 \cdot 1 = 16$	$l_{\text{acc}} = l_p + \lceil \log_2 16 \rceil = 2 + 4 = \mathbf{6}$

Resulting typedefs:

```
// W chosen by sweep (see
// next slide)
typedef ap_fixed<W, 1>
    data_t; // input
typedef ap_fixed<W, 1>
    coef_t; // coeff
typedef ap_fixed<W+4,6>
    acc_t; // accum
```

The accumulator needs 4 extra bits relative to the data type. This is the guard-bit rule: $\lceil \log_2 N \rceil$ for N additions.

FIR Implementation in ap_fixed

```
#include "ap_fixed.h"
#define N_TAPS 16
typedef ap_fixed<12, 1, AP_RND, AP_SAT> data_t;
typedef ap_fixed<12, 1, AP_RND, AP_SAT> coef_t;
typedef ap_fixed<16, 6, AP_RND, AP_SAT> acc_t;
void fir_fixed(data_t x[N_TAPS], coef_t h[N_TAPS], data_t *y)
{
    #pragma HLS PIPELINE II=1
    #pragma HLS ARRAY_PARTITION variable=x complete
    #pragma HLS ARRAY_PARTITION variable=h complete
    acc_t acc = 0;
    for (int k = 0; k < N_TAPS; k++) {
    #pragma HLS UNROLL
        acc += (acc_t)(h[k] * x[k]);
    }
    *y = (data_t)acc;
}
```

ARRAY_PARTITION complete: 16 parallel MACs $\Rightarrow$ II=1. Cast (data_t)acc makes narrowing explicit.

SNR vs. W — Testbench Results

W	Frac bits	LSB	SNR (dB)	Meets 60 dB?
8	7	0.0078	44	No
10	9	0.0020	56	No
12	11	0.0005	68	Yes — min. W
16	15	3×10^{-5}	92	Yes
20	19	2×10^{-6}	116	Yes

- ▶ $W = 12$: **minimum** passing 60 dB target
- ▶ SNR +6 dB per extra bit — matches theory
- ▶ $W = 8$ fails: only 7 frac. bits at $I = 1$

Result: `ap_fixed<12,1>` for data;
 `ap_fixed<16,6>` for accumulator.

FIR Synthesis Results — W Sweep

W	LUTs	DSPs	II	Note
8	80	16	1	Fails SNR target
12	130	16	1	Chosen design
16	165	16	1	Overkill for 60 dB
20	220	32	1	DSP usage doubles

- ▶ $II = 1$ is maintained for all W up to 20 on the Zynq-7020
- ▶ At $W = 20$, DSP usage doubles because 20-bit multipliers no longer fit in a single DSP48; HLS maps to two
- ▶ $W = 12$ is the **knee of the curve**: minimum area, passes SNR, $II = 1$, fits comfortably in the Zynq-7020

Illustrative figures for a 16-tap FIR on Zynq-7020 at 100 MHz.

Float vs. `ap_fixed<12,1>` — Final Comparison

Metric	<code>float32</code> FIR	<code>ap_fixed<12,1></code> FIR
LUTs	~820	130 (6× fewer)
DSPs	32	16 (2× fewer)
II	4	1 (4× faster)
SNR	149 dB	68 dB (meets 60 dB target)
Verdict	Accurate, expensive, slow	Accurate enough, lean, fast

Key insight: `float32` gives 149 dB SNR — more than 80 dB beyond the 60 dB requirement. That surplus accuracy is being paid for in area and throughput. `ap_fixed<12,1>` recovers that cost while still exceeding the target.

L7 Key Takeaways

1. **Methodology:** range $\rightarrow$ target $\rightarrow$ sweep $W \rightarrow$ check II & area. Follow all 4 steps every time.
2. **Accumulator rule:** $W_{\text{acc}} = W_{\text{data}} + \lceil \log_2 N \rceil$; $I_{\text{acc}} = I_{\text{data}} + \lceil \log_2 N \rceil$. Never guess — derive it.
3. **Measure SNR:** run a C testbench with real inputs. Theory gives you a starting point; simulation gives you the answer.
4. **ap_fixed vs. float:** fixed-point gives $II=1$ and far lower area. Choose the smallest W that meets your SNR target.

ap_fixed Quick Reference

Template

```
ap_fixed<W, I, Q, O>  
    // W : total bits  
    // I : integer bits (incl.  
        sign)  
    // Q : AP_TRN (default) /  
        AP_RND  
    // O : AP_WRAP (default) /  
        AP_SAT
```

Width rules

```
// After N MACs:  
W_acc = W_data + ceil(log2(N))  
I_acc = I_data + ceil(log2(N))  
// Multiplier output:  
W_out = W_A + W_B
```

Common type recipes

Use case	Type
Audio $[-1, 1]$	$\langle 16, 1, \text{RND}, \text{SAT} \rangle$
Image $[0, 1]$	$\langle 8, 0, \text{RND}, \text{SAT} \rangle^{\dagger}$
FIR coeff	$\langle 12, 1, \text{RND}, \text{SAT} \rangle$
FIR accum ($N=16$)	$\langle 16, 5, \text{RND}, \text{SAT} \rangle$
GEMM accum ($n=64$)	$\langle 24, 8, \text{RND}, \text{SAT} \rangle$

[†] Use `ap_ufixed` for unsigned.

Always check for overflow warnings in the HLS synthesis report after any type change.

What's Next — L8 Preview



— Memory Hierarchy & Cache

- ▶ **OCM vs DDR:** on-chip memory (OCM, 256 kB) is $\approx 100\times$ faster than DDR — knowing when your data fits in OCM matters
- ▶ **L1/L2 cache behaviour:** how the Cortex-A9 caches interact with your bare-metal loops; cache-friendly access patterns
- ▶ **Cache coherency with PL:** why `Xil_DCacheFlushRange` and `Xil_DCacheInvalidateRange` are mandatory around every DMA transfer

Lab note: for your projects, try replacing `float` with `ap_fixed` in your HLS kernel and measure the area and throughput improvement.

Questions?

Range
→ / bits

Target
SNR

Sweep
 W

Check
resources